

I.- Introducción.

Tradicionalmente la computadora ha sido vista como una máquina secuencial, los programadores escribían algoritmos como secuencias de instrucciones y las CPU's las ejecutaban en secuencia una tras otra. Aunque esto, funcionalmente hablando, no ha sido nunca del todo cierto (activación simultánea de señales de control en microoperaciones, segmentación, superescalaridad, etc.), si es cierto que con la evolución tecnológica de los computadores, tanto en hardware como en software, se han desarrollado sistemas en los que se puede disponer de múltiples CPU's para la ejecución de programas.

A estos sistemas es a los que se hace referencia cuando se habla de paralelismo y/o multiprocesamiento. Podemos hacer una primera clasificación de estos sistemas en función de lo relacionadas que estén las CPU's sería:

- Multiprocesadores estrechamente acoplados, o de memoria compartida:

Se trata de un conjunto de procesadores que comparten una memoria principal común y se encuentran bajo el control integrado de un mismo sistema operativo.

- Multiprocesadores ligeramente acoplados, o de memoria distribuida:

Consiste en una colección de sistemas relativamente autónomos en los que cada CPU tiene su propia memoria principal y canales de E/S.

- Procesadores funcionalmente especializados:

Son sistemas en los que normalmente existirá una CPU de propósito general, maestra, y procesadores especializados que son controlados por la CPU maestra y le proporcionan servicios a ésta.

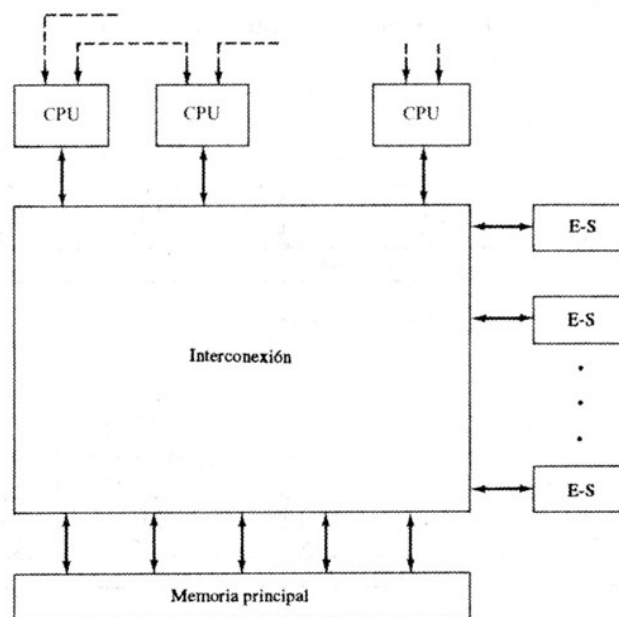
Desde el punto de vista de arquitectura nos interesa estudiar los sistemas multiprocesadores estrechamente acoplados, que normalmente denominamos sistemas multiprocesador.

II.- Generalidades Multiprocesadores de memoria compartida.

Los multiprocesadores se caracterizan por:

- 1.- Disponer de dos o más procesadores similares de propósito general y de capacidad comparable.
- 2.- Todos los procesadores comparten accesos a la memoria global (común), pudiendo disponer cada uno de ellos de memoria local (privada).
- 3.- Todos los procesadores comparten accesos a todos los dispositivos de E/S a través de *redes de interconexión*.
- 4.- El sistema multiprocesador está controlado por un sistema operativo integrado que proporciona interacción entre los procesadores y sus programas en los niveles de elementos de trabajo, tarea, archivo y datos. Este punto es el que más claramente distingue los sistemas estrechamente acoplados de los ligeramente acoplados.

En términos generales, la organización de un sistema multiprocesador puede representarse de la siguiente forma:



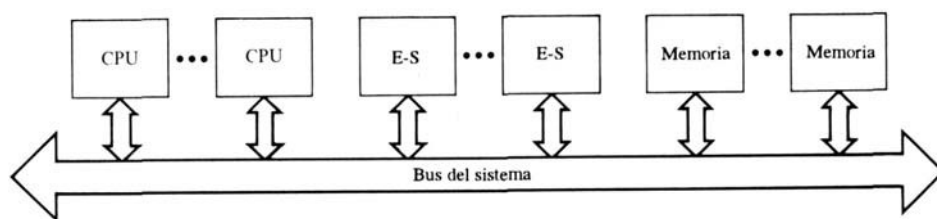
Arquitecturas Paralelas

Donde existen dos o más CPU's con acceso a una memoria principal compartida y a los dispositivos de E/S mediante una red de interconexión. Los procesadores pueden comunicarse entre sí a través de la memoria, usando áreas de datos comunes, o intercambiando señales de forma directa. Existen casos en los que cada CPU puede disponer de su propia memoria principal y canales de E/S privados.

- Clasificación según acceso a recursos compartidos:

Los sistemas multiprocesadores pueden organizarse de forma genérica y desde el punto de vista de configuración del acceso a recursos compartidos en alguno de los siguiente grupos:

- a) Sistemas de tiempo compartido o bus común.



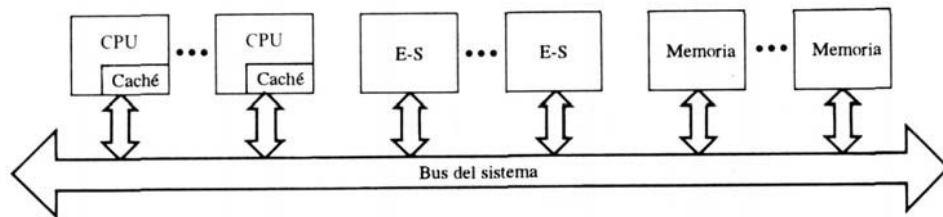
Es el mecanismo más simple y su estructura responde básicamente a la de un sistema monoprocesador que utilice una interconexión de bus. El bus consiste en líneas de control, de direcciones y de datos que ofrecen características de:

- Direccionamiento: Distinguir módulos en el bus para identificar fuente y destino de datos.
- Arbitraje: Mecanismo para resolver solicitudes que compiten por el control de bus. Cualquier módulo podría funcionar temporalmente como maestro, utilizando normalmente algún esquema de prioridades.
- Compartición en tiempo: El control del bus pasa a manos del módulo maestro, y el resto de módulos quedarían bloqueados debiendo suspender su operación, si fuera necesario, hasta que logran acceso al bus.

Las ventajas de este tipo de sistemas es su simplicidad, flexibilidad y confiabilidad frente a otras propuestas.

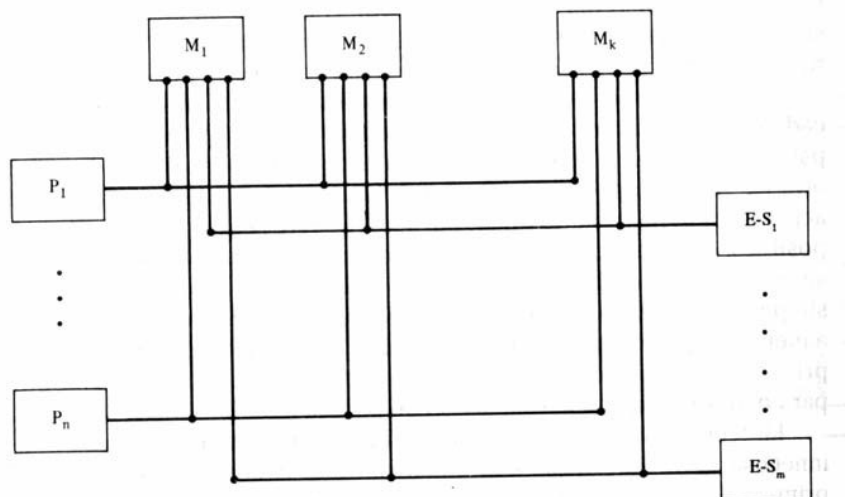
Arquitecturas Paralelas

El principal inconveniente es el desempeño, todas la referencias pasan a través del bus limitando bastante la velocidad del sistema. Una posible mejora a ésto es el uso de memorias cache locales a cada CPU, lo que introduce otra problemática o consideraciones de diseño como el tipo de escritura en memoria principal o mecanismos para mantener la coherencia de los datos en cache.



b) Sistemas de memoria multipuertos.

Esta propuesta permite acceso independiente a módulos de memoria principal y de E/S por parte de cada CPU.



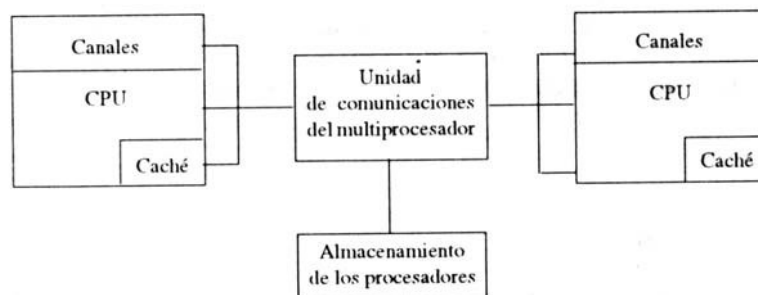
Arquitecturas Paralelas

En este caso se requiere lógica adicional para resolver los conflictos de acceso a memoria, normalmente se resuelven asignando permanentemente prioridades a cada puerto de memoria por parte de las CPU's. Este es el principal inconveniente respecto al bus, la complejidad de la lógica adicional necesaria en el sistema de acceso a memoria.

Las principales ventajas respecto al bus son la mejora del desempeño en el acceso a memoria al disponer de trayectorias dedicadas para cada módulo de memoria y el poder disponer de memoria y/o módulos de E/S dedicados o privados a una o más CPU's.

c) Sistemas con unidad de control central.

Disponer de una unidad de control central de comunicaciones permite encauzar los flujos de datos entre módulos independientes (CPU's, memoria y E/S) en la red de interconexión. Este controlador podrá almacenar solicitudes y realizar funciones de temporización y arbitraje entre dichos módulos, así como paso de mensajes de control y estado entre CPU's (paralelismo de tareas, datos, actualizaciones de cache, etc.).



Esta organización tiene las ventajas del bus y memoria multipuertos, y los inconvenientes que suponen el cuello de botella potencial en cuanto a desempeño y rapidez de ejecución del controlador, y la complejidad de la lógica necesaria por dicha unidad de control central.

Arquitecturas Paralelas

- Clasificación según flujo de datos e instrucciones:

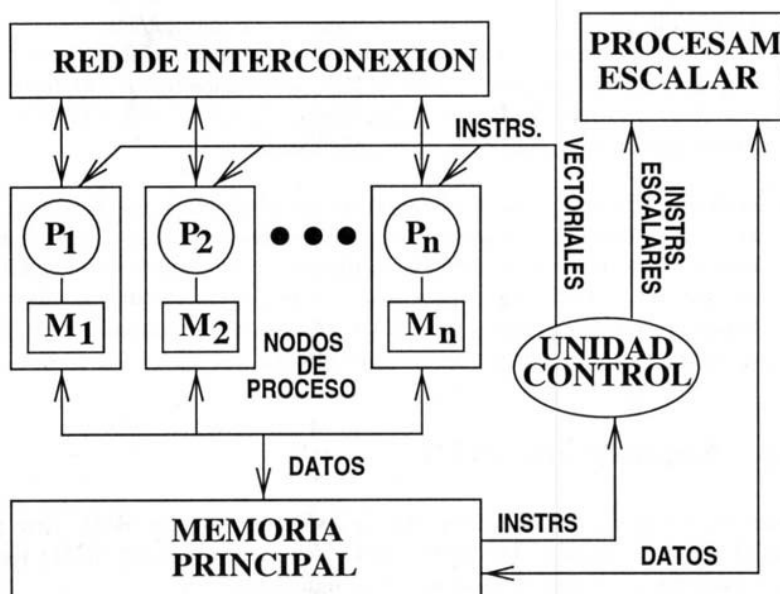
Otra clasificación de sistemas multiprocesadores (supercomputadores) puede establecerse desde el punto de vista de arquitectura interna y flujo de datos e instrucciones. La gran variedad de arquitecturas paralelas que han surgido en los últimos años pretenden velocidades de computación pico muy elevadas y arquitecturas escalables a un gran número de procesadores.

Se destacan cuatro tipos de supercomputadores, dos con desarrollos comerciales que se detallan más adelante (SIMD y MIMD), y otros dos que raramente se implementan como son SISD y MISD.

- SIMD: Simple flujo de instrucciones, múltiple flujo de datos. Una sola instrucción se aplica simultáneamente sobre muchos datos.
- MIMD: Múltiple flujo de instrucciones, múltiple flujo de datos. Máquinas con varias unidades de procesamiento en las que múltiples instrucciones pueden actuar sobre diferentes conjuntos de datos de forma simultánea, es decir, cada procesador puede estar ejecutando un programa distinto al mismo tiempo. Las MIMD son las máquinas más complejas y las que mejores prestaciones ofrecen.

II.1.- Arquitecturas SIMD.

Todos los computadores SIMD tienen una estructura semejante, reflejo de la poca flexibilidad de diseño que supone la necesidad de sincronización de instrucciones mediante señales de control.



Arquitecturas Paralelas

Un computador SIMD se compone de un conjunto de nodos de procesamiento (PN's) y un procesador escalar, que operan bajo las órdenes de una unidad de control desde donde se centraliza el funcionamiento de todo el sistema.

La unidad de control busca y decodifica instrucciones de la memoria principal y, dependiendo de su tipo, envía las correspondientes señales de control al procesador escalar o a los nodos de procesamiento para su ejecución.

Si se trata de una instrucción escalar, sólo funcionará el procesador escalar; en caso contrario, funcionarán todos los nodos de procesamiento en paralelo, los cuales ejecutarán la misma instrucción sobre datos diferentes almacenados en sus respectivas memorias locales. Por ejemplo, si consideramos el programa de suma de dos vectores de cien datos A y B, almacenando el resultado en otro vector C de cien posiciones; podríamos tener cien flujos de datos distintos, uno para cada operación suma y realizar todas ellas a la vez en un tiempo de ejecución cien veces más rápido.

Todos los nodos de procesamiento del computador SIMD son controlados desde el mismo programa sin necesidad de copiar el código del programa en cada uno de ellos. Es decir, se trata de explotar el paralelismo que presenta el conjunto de datos de una aplicación en lugar de paralelizar la secuencia de ejecución de las instrucciones que la componen.

Desde el punto de vista de la eficiencia, el acceso a un dato no local es mucho más lento, y por tanto, la paralelización de una aplicación debe de procurar descomponer los datos del problema entre las memorias locales de los procesadores de forma que se maximice el número de referencias a memoria local.

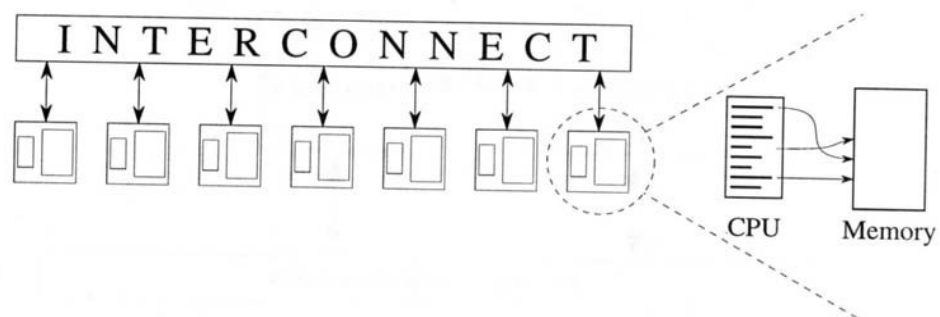
Cuando un nodo de procesamiento realiza una operación sobre un conjunto de datos que no están alojados en su totalidad en su memoria local, la unidad de control debe encargarse de transferir esos datos a través de la red de interconexión desde la memoria local del procesador que los contenga.

II.2.- Arquitecturas MIMD.

En función del modo en que se conectan los procesadores y los módulos de memoria principal podemos distinguir entre máquinas de memoria compartida y máquinas de memoria distribuida, clasificando estas arquitecturas en distintos subgrupos:

a) Arquitecturas de memoria distribuida:

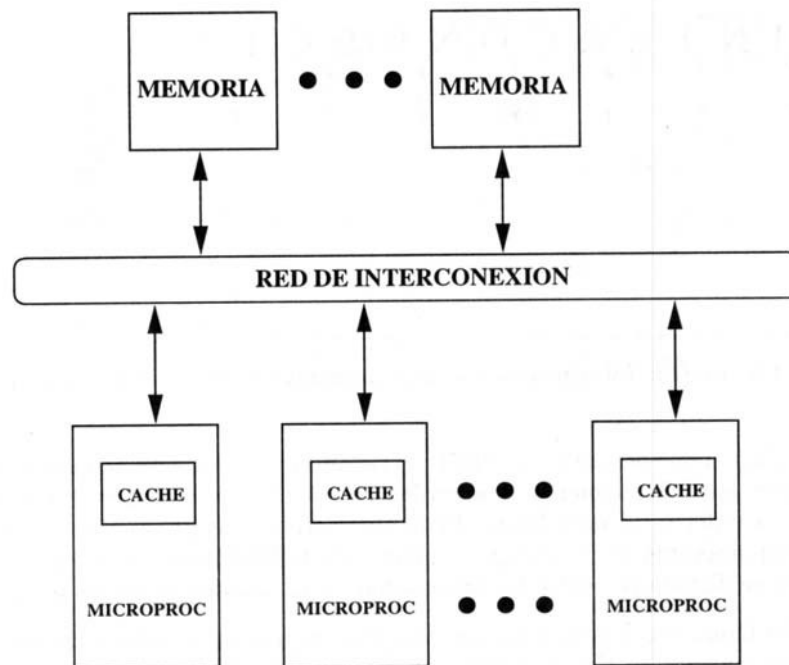
Una red de computadores constituye el esquema básico de una máquina de memoria distribuida, en la que cada procesador dispone de su propia memoria y se comunica con los demás a través de la red de interconexión.



Podríamos encontrar cierta similitud con una arquitectura SIMD a nivel interno de cada nodo de procesamiento en cuanto a disponer de un procesador y memoria. Sin embargo, en este caso cada nodo dispone de su propia unidad de control o microprocesador, lo que permite al sistema ejecutar un programa diferente en cada uno de sus nodos de forma simultánea.

Como ocurre con los SIMD, el tiempo requerido para acceder a un dato no local es aproximadamente de un orden de magnitud mayor que el que se necesita para acceder a un dato local. El rendimiento de los programas sobre este tipo de arquitectura se encuentra así igual y enormemente influenciado por la forma en que los datos se distribuyen entre los procesadores y se acceden desde los programas.

b) Arquitecturas de memoria compartida:



Se trata de conectar todos los procesadores y módulos de memoria a una red de interconexión de forma que se cree un único espacio de direcciones común a todos los procesadores. Una misma posición de memoria representa ahora la misma celda física de almacenamiento en todos los procesadores y tarda el mismo tiempo en ser accedida desde cualquier procesador. De esta forma, los programadores de este tipo de máquinas no necesitan enviar datos entre las CPU's, ni indicarlo explícitamente mediante instrucciones de comunicación send/receive. La gran ventaja de la memoria compartida es por tanto su facilidad de programación.

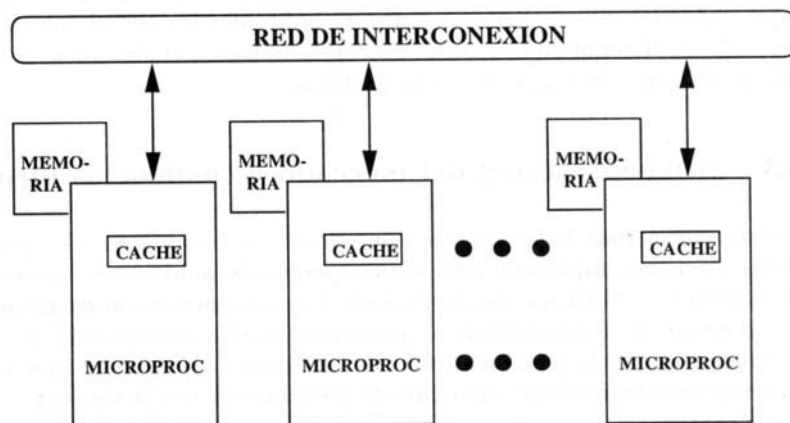
El principal inconveniente de esta arquitectura es su pobre escalabilidad. Esto es debido a que a partir de una docena de procesadores la red de interconexión no puede absorber el elevado número de comunicaciones que producen las peticiones a memoria generadas desde los procesadores. Esta desventaja es mayor cuanto peor sea la configuración de la red de interconexión, como sucede con el bus común.

Arquitecturas Paralelas

Una mejora introducida en estos sistemas es el concepto de localidad en el acceso a datos, si a este tipo de arquitecturas se las provee de memorias cache locales a cada procesador. Estas memorias tratan de retener los datos de memoria más usados con el fin de disminuir el tráfico que soporta la red de interconexión, y lo hacen en mayor medida cuanto mayor es la velocidad del algoritmo paralelo.

c) Arquitecturas de memoria compartida/distribuida:

Es la aproximación más avanzada y la que adoptaron los últimos supercomputadores lanzados al mercado. Trata de combinar las ventajas de los dos modelos anteriores, o sea, la escalabilidad del modelo distribuido y la facilidad de uso del modelo compartido. Este tipo de arquitecturas puede esquematizarse de la siguiente forma:



Donde vemos que la memoria está físicamente distribuida entre los procesadores, pero un nivel hardware intermedio de enrutado se encarga de realizar las comunicaciones de forma automática y casi transparente al usuario. A este tipo de arquitecturas se las denomina NUMA (Non-Uniform Memory Architecture), dado que el acceso a un módulo de memoria es directo desde su procesador local mientras que debe hacerse a través de la red de interconexión desde cualquier otro procesador.

Arquitecturas Paralelas

La forma de programar estos sistemas es similar a la de un computador de memoria compartida, ya que a nivel lógico o de usuario existe un solo espacio de direcciones. La única diferencia sustancial es que aquí suele exigirse al usuario que distinga entre variables privadas (sólo las puede usar el código de un procesador) y variables compartidas (pueden ser accedidas por todos). Esto suele hacerse para mejorar la disposición de los datos en los diferentes módulos de memoria con el fin de optimizar la localidad de acceso a los datos, aspecto que sigue siendo la clave para el buen rendimiento de este tipo de máquinas.

Las máquinas de memoria compartida/distribuida presentan una mejor escalabilidad comparadas con el modelo compartido puro, pero no tan buena como la del caso distribuido debido a las limitaciones del hardware de enrutado para resolver eficientemente un alto volumen de peticiones no locales.

III.- Organización de memoria paralela.

- INTRODUCCIÓN.

Técnicas de diseño de memorias para multiprocesadores

Método entrelazado: Extensión de las técnicas aplicadas a procesadores segmentados y vectoriales.

Problemas presentados con múltiples cachés privadas. Soluciones.

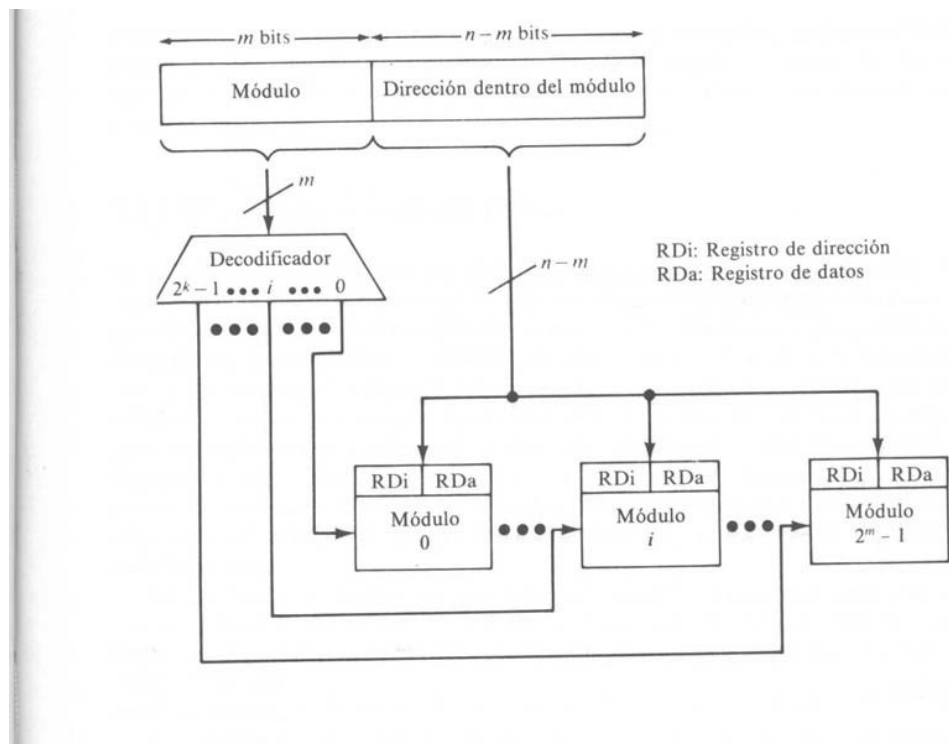
- MEMORIA ENTRELAZADA.

Particionar la memoria en varios módulos.

Distribuir las direcciones entre dichos módulos.

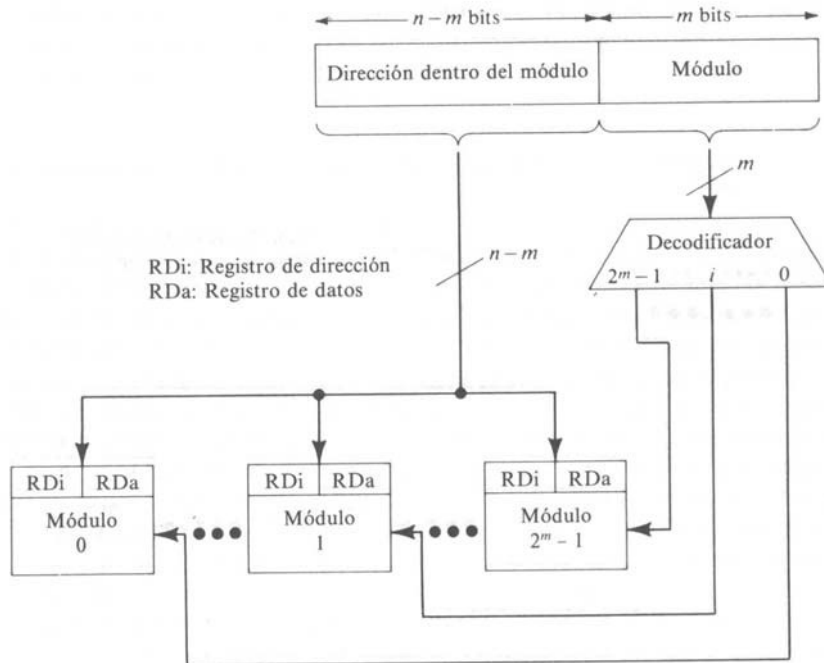
Resuelve interferencias al permitir accesos concurrentes a más de un módulo.

- Métodos de distribución de direcciones:
 - Entrelazado de orden superior: Los bits de orden superior seleccionan el módulo y los de orden inferior la dirección dentro del módulo. Mayor fiabilidad.



Arquitecturas Paralelas

- De orden inferior: Los bits de orden inferior seleccionan el módulo y los de orden superior la dirección (direcciones consecutivas en módulos consecutivos).



- CONFIGURACIONES DE MEMORIA ENTRELAZADA.

Memoria particular "Mi" del procesador "i".

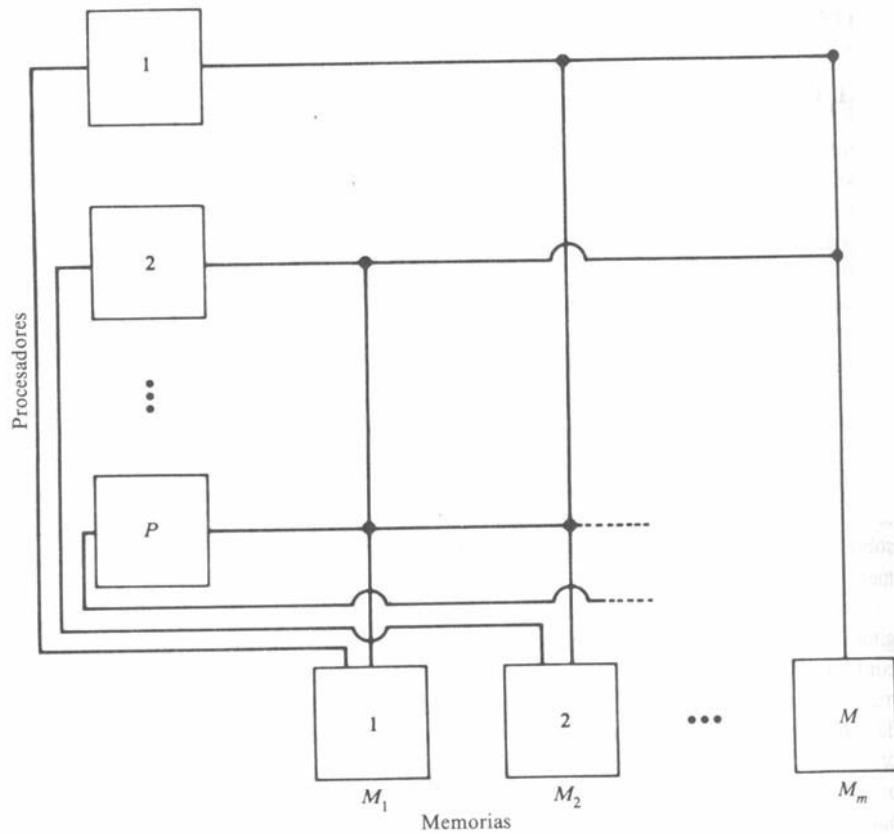
Asignar páginas del proceso a ejecutar "i" al módulo de memoria "Mi", con entrelazado superior.

Reducción de interferencias de memoria.

Extensión del concepto de memoria particular. Más módulos de memoria que número de procesadores.

Arquitecturas Paralelas

- Ejemplo de organización: Dos puertos por módulo de memoria (uno a PMIN, y otro al procesador "local").



Ventajas:

- Aumentar el número de accesos de cada procesador a su memoria particular, sin pasar por la PMIN
- La participación de la PMIN se hace menos crítica al ser menos usada.
- Mayor fiabilidad en la operación del sistema. Un fallo en memoria invalida sólo el trabajo con ese módulo de memoria.

Arquitecturas Paralelas

- Memoria con acceso concurrente: Los procesadores y la memoria principal se encuentran en lados opuestos de la PMIN

Las referencias a memoria deben atravesar la PMIN. Inconvenientes: Conflictos de memoria y retardos de transmisión. Reducir inconvenientes: Cache privada asociada a cada procesador.

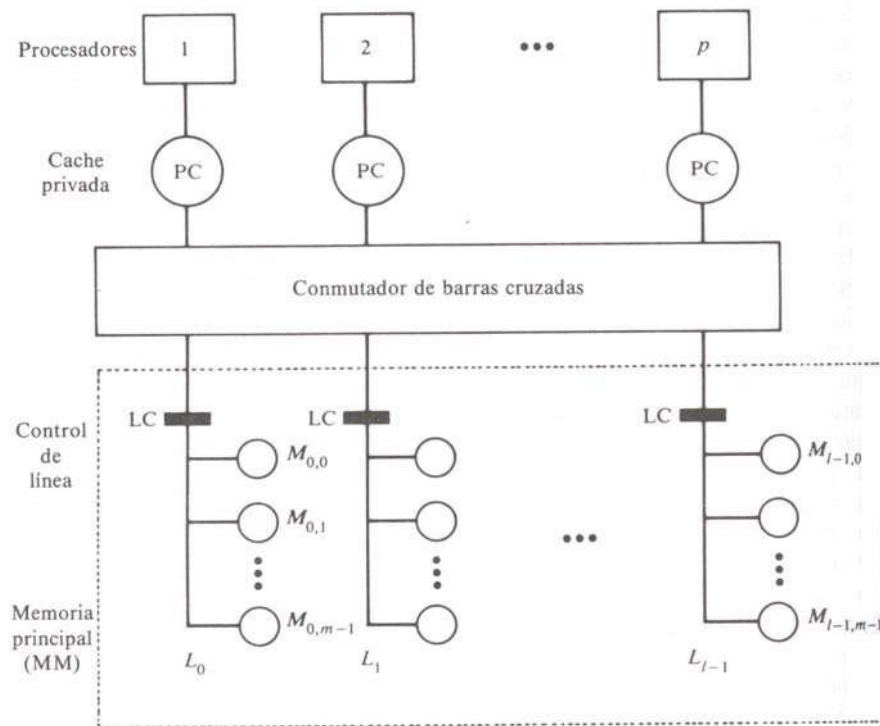
Problema: anchura del bus de datos, puede afectar al coste y tiempo de transferencia de un bloque de datos.

Ejemplo: Ancho de banda de un módulo de memoria de 8 bytes y 4 módulos de memoria con acceso concurrente, se pueden transferir 32 bytes en poco más de un ciclo de memoria. Un computador con un bloque cache de 32 bytes puede transferir este bloque en ese tiempo.

Llegada escalonada de las 4 palabras a la cache.

Arquitecturas Paralelas

- Ejemplo de organización L-M (memoria bidimensional): “l” líneas de acceso concurrente con “m” módulos por línea.



Ventaja: Mayor flexibilidad. Inconvenientes:

Inconvenientes:

Degradación del rendimiento por compartición de líneas y módulos.

Conflictos: Varias peticiones al mismo módulo; bloqueo del procesador por petición a un módulo ocupado.

- SISTEMAS MULTICACHE.

Sistemas con caches asociadas a los procesadores.

Caches privadas en un multiprocesador introduce problemas de coherencia cache y de inconsistencia de datos. La inconsistencia se produce cuando existen varias copias del mismo dato en diferentes caches en un mismo instante.

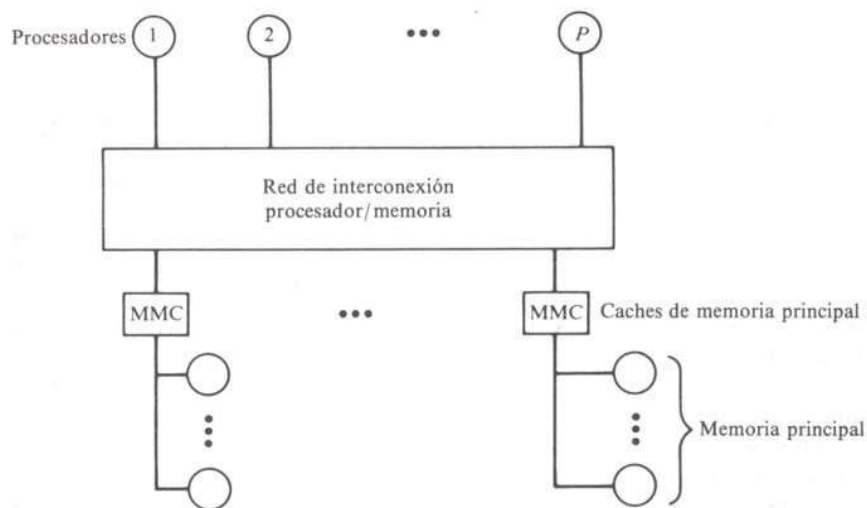
Ejemplo: Proceso A en procesador i produce dato x que es usado por el proceso B en el procesador j. El proceso B debe ser informado de la presencia del nuevo dato x en la cache del procesador i.

Cambio de contexto en sistemas multiprocesadores multiprogramados. Limpiar cache.

Sistema de caches es coherente si y sólo si la lectura de una posición de memoria x, por el procesador i, extrae siempre el valor más reciente de esa posición x.

- Soluciones:

Asociar las caches a las líneas de memoria compartida.



Buena opción para sistemas con pocos procesadores.

Velocidad limitada por retardos de transmisión y conflictos de acceso.

Arquitecturas Paralelas

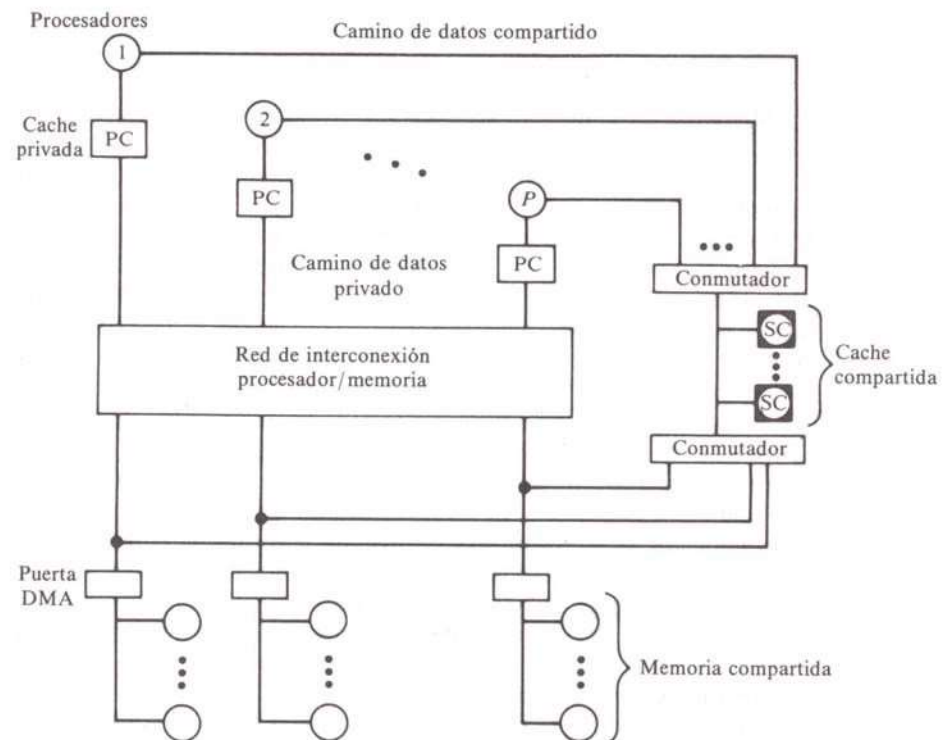
Problema de coherencia cache no simplemente solucionable con políticas de escritura directa (write through), al existir copias en caches locales.

- Control de coherencia estático.

Evitar múltiples copias de un dato.

Implementa caminos diferentes para datos modificables compartidos (no soportables en cache) y datos privados (soportables en cache).

Posibilidad de soportar en cache datos de sólo lectura.



- Control de coherencia dinámico.

Más flexible que el estático. Más complejo y costoso.

Se permiten múltiples copias de un dato.

Arquitecturas Paralelas

Si un procesador modifica una posición x en un bloque cache debe controlar a las otras caches para invalidar posibles copias. Interrogación cruzada, XI (Cross Interrogate).

- Caso de implementación: Caches unidas por un bus de alta velocidad.

Cuando escribe en un bloque de su cache, el procesador envía una señal a todas las caches remotas indicando que el dato x ha sido modificado, al mismo tiempo que se escribe en memoria principal.

No puede terminar la escritura hasta que la XI no recibe contestación de todos.
Producirá fallo en cache en los procesadores remotos que accedan al dato x .
Indicación de presencia, peticiones filtradas antes de ser iniciadas.

Existen varias tablas centrales asociadas con los bloques de la memoria principal en las que hay marcas para indicar la presencia o no del dato y su validez.

Dentro de cada cache existen indicadores locales de estado de cada bloque en la cache.
Política de actualización de postescritura (write-back).

Las fuentes principales de degradación del rendimiento en el control de coherencia dinámico son:

Invalidaciones de bloque: degradan tasa media de acierto.
Tráfico entre caches (asegurar consistencia).
Conflictos por accesos concurrentes a tablas globales.
Reescritura debida a invalidación de datos.

Arquitecturas Paralelas

- Localidad cache.

Sistemas multiprocesadores estrechamente acoplados de memoria compartida/distribuida (arquitecturas NUMA): muy sensibles al uso apropiado de la jerarquía de memoria.

Solución: Explotar la localidad de las referencias a memoria.

Minimizar el número de accesos a posiciones de memoria remota.

Seleccionar distribuciones (iteraciones y datos) que faciliten el alojamiento de información en las memorias locales de los procesadores que la demanden.

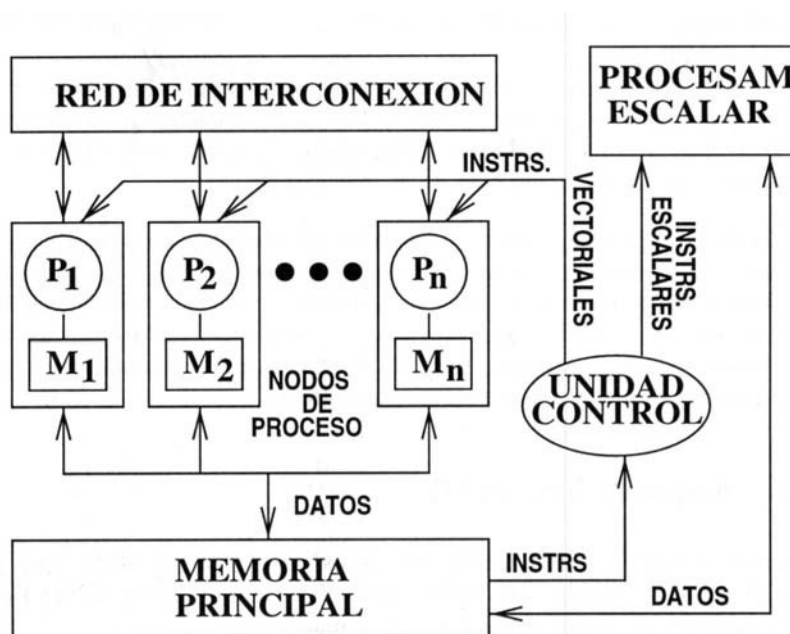
Importancia de la automatización de la distribución.

IV.- Redes de interconexión.

Las redes de interconexión permiten la comunicación entre los distintos módulos que forman el sistema multiprocesador. Según el tipo de arquitectura que tratemos SIMD o MIMD tendremos una problemática distinta en la interconexión de dichos módulos o unidades que forman el sistema, distinguimos por tanto dos tipos de redes de interconexión, las SIMD y las MIMD.

IV.1.- Redes de interconexión SIMD y MMD.

Las primeras responden a la interconexión necesaria entre “nodos de procesamiento” de los computadores SIMD, como muestra la siguiente figura:



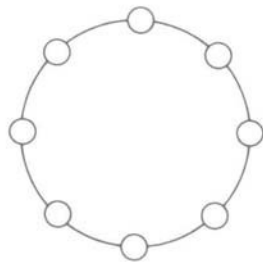
Además, se distinguen entre redes estáticas y redes dinámicas; y entre redes monoetapa y redes multietapa.

Arquitecturas Paralelas

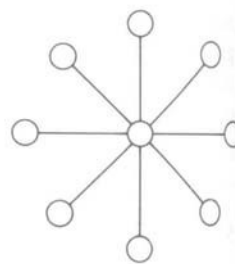
- Redes estáticas: Los enlaces entre dos procesadores son pasivos y los buses dedicados no pueden ser reconfigurados para establecer conexiones directas a otros procesadores. Según las dimensiones que requiera su estructura podemos distinguir las siguientes topologías:
 - Topologías unidimensionales. Red lineal para arquitecturas encauzadas.



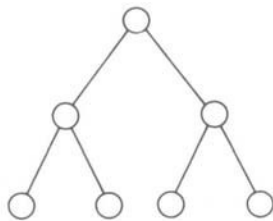
- Topologías bidimensionales. En anillo, estrella, árbol, malla y red sistólica.



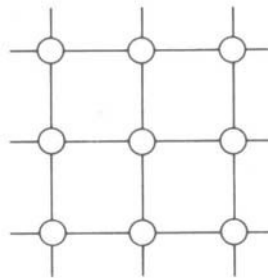
Anillo



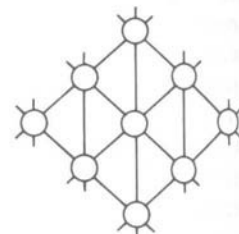
Estrella



Arbol



Malla

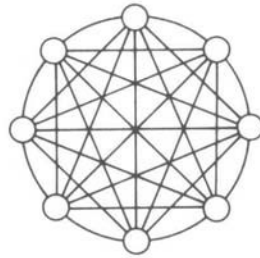


Red sistólica

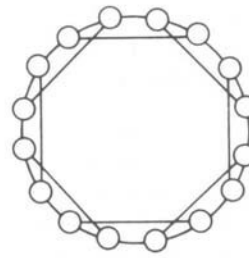
Actualmente, utilización muy frecuente de topología en estrella. Computadores conectados mediante hub o concentrador. Ventajas: Menor impacto de fallos de computadoras, gestión centralizada, fácil administración y mantenimiento, mayor rendimiento y seguridad. Desventajas: dependencia y cuello de botella del dispositivo concentrador, costo de despliegue.

Arquitecturas Paralelas

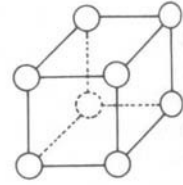
- Topologías tridimensionales. Red cordal, de conexión completa, cubo-3 y cubo-3 ciclo-conexa.



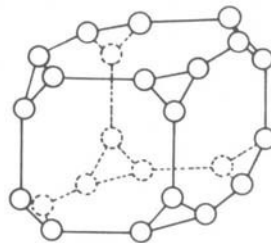
Totalmente conexa



Anillo cordal



Cubo-3

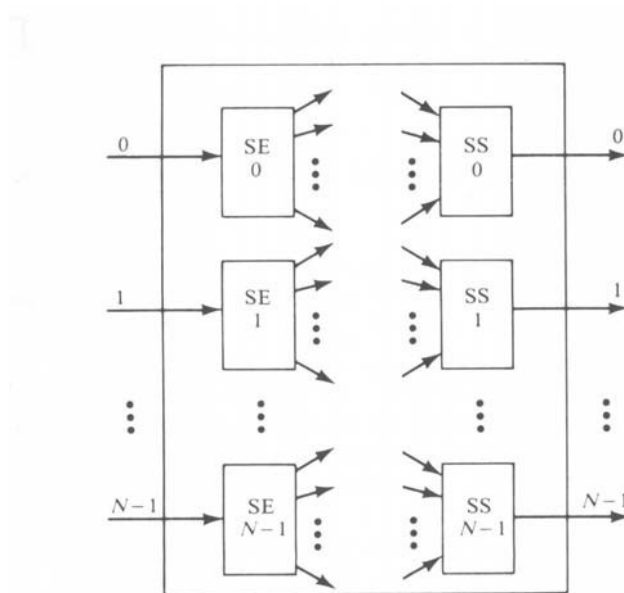


Cubo-3 ciclo-conexo

- Topologías hipercubo. Un hipercubo D-dimensional de ancho A, contiene A nodos en cada dimensión y dispone de una conexión a un nodo en cada dimensión. La malla y el cubo-3 son realmente hipercubos bi y tridimensionales.
- Redes dinámicas: Pueden ser reconfiguradas mediante el ajuste de elementos activos de conmutación de la red. Se distinguen dos tipos de redes dinámicas, monoetapa y multietapa.

Arquitecturas Paralelas

- Redes monoetapa. Es una red de conmutación con N selectores de entrada (SE) y N selectores de salida (SS) como se indica a continuación.



Cada SE consiste básicamente en un demultiplexor de 1-a-D y cada SS en un multiplexor de M-a-1, donde $1 \leq D \leq N$ y $1 \leq M \leq N$. Para establecer un camino de datos deseado se han de aplicar diferentes señales de control a todos los selectores SE y SS. En el caso de que $D = M = N$ tendremos una red de conmutación de barras cruzadas.

Las redes monoetapa se denominan también recirculantes, ya que los ítems de datos pueden tener que circular varias veces por la red antes de alcanzar sus destinos finales. El número de recirculaciones depende de la conectividad de la red, a mayor conectividad menor número de recirculaciones. La red de barras cruzadas es un caso extremo en el que sólo es necesaria una circulación para establecer cualquier comunicación entre dos NP; el inconveniente que presentan es el coste que supone la

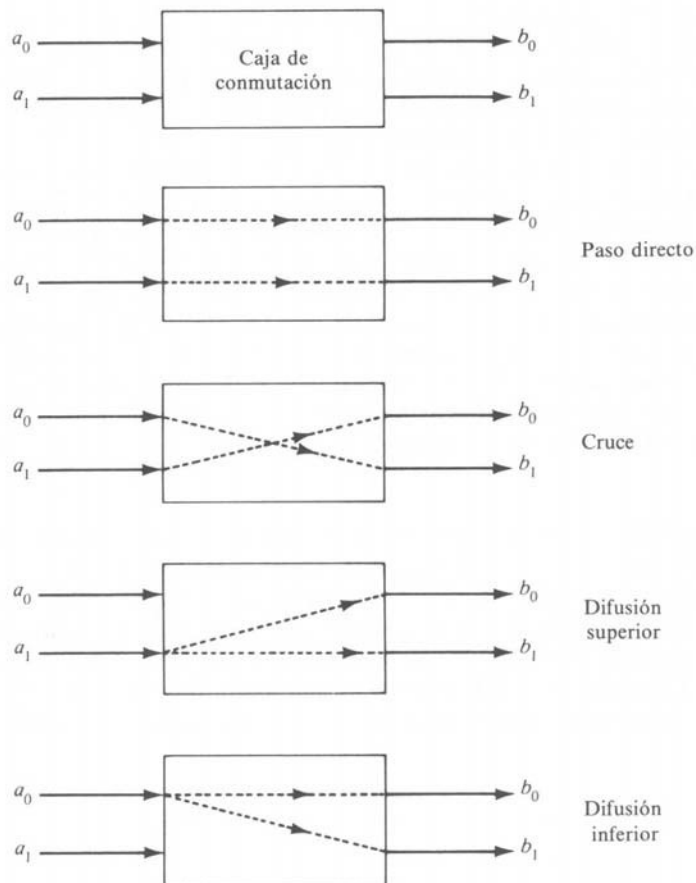
conectividad total, del orden de N^2 ($O(N^2)$). Para valores grandes de N la

mayoría de las redes recirculantes tendrán un coste de $O(N \log N)$, que supone una mejor relación coste-efectividad.

Arquitecturas Paralelas

- Redes multietapa. Existen 3 características que definen las redes multietapa, son: la caja de conmutación, la topología de red y la estructura de control.

Las cajas de conmutación son dispositivos de intercambio de dos entradas y dos salidas, distinguiéndose principalmente cuatro posibles estados en ellas: paso directo, cruce, difusión superior y difusión inferior.

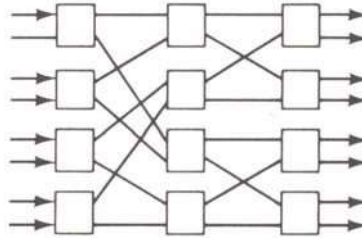


Las cajas de conmutación de las redes multietapa pueden ser de entrada y salida en el mismo lateral y de sólo de entrada o solamente salida, éstas últimas pueden ser de bloqueo, reordenables y sin bloqueo.

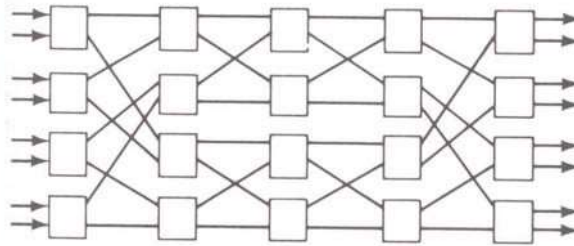
Las redes con bloqueo son aquellas en las que se pueden producir conflictos en el uso de los enlaces de la red de comunicación en caso de que se esté produciendo la conexión simultánea de más de una pareja de terminales.

Arquitecturas Paralelas

Las redes reordenables son las que pueden efectuar todas las conexiones posibles entre las entradas y las salidas mediante la reordenación de sus conexiones; siempre habrá una vía de conexión entre una pareja entrada-salida.



La red de Benes para permutación síncrona de datos o comunicación asíncrona de procesadores.



Una red sin bloqueos es aquella que puede manejar todas las posibles conexiones sin que se produzcan bloqueos, pueden conectar cualquier puerto de entrada a un puerto de salida libre. Ejemplos de este tipo son la red de Clos y la de barras cruzadas (crossbar).

Las redes multietapa suelen constar de “n” etapas con “N” líneas de entrada y salida, donde $N=2^n$. Cada etapa suele utilizar $N/2$ cajas de conmutación y está conectada a la siguiente por al menos N caminos. El retardo de la red es proporcional al número de etapas, y el coste de la red es proporcional a $N \cdot \log_2 N$.

Los esquemas de interconexión de etapa a etapa definen la topología de la red.

La estructura de control determina cómo se pueden fijar los estados de las cajas de conmutación, y pueden ser: control por etapas, donde la misma señal de control se fija para todas las cajas de la misma etapa; control individual por caja, existe una señal de control separada para fijar el estado de cada caja individual. Ésta última opción ofrece mayor flexibilidad pero requiere un coste mucho más elevado. Existen posibilidades intermedias que ofrecen control parcial por etapas.

Arquitecturas Paralelas

Ejemplos de distintas configuraciones de este tipo de red son los siguientes:

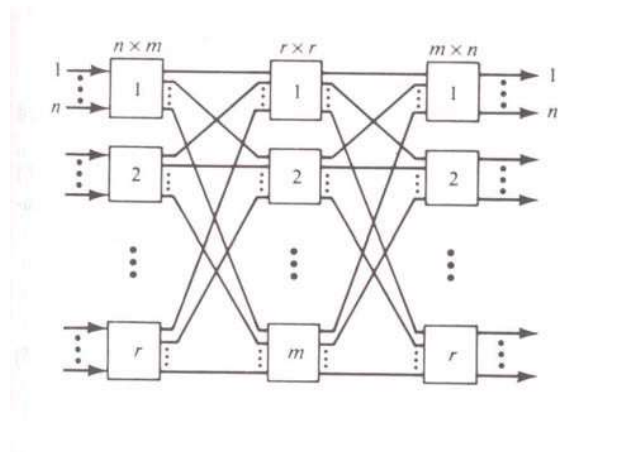
- Red Illiac conectada en malla:

Se trata de una red recirculante monoetapa implementada en el procesador matricial Illiac-IV con $N=64$ nodos procesadores. En este caso cada NP está

conectado a otros 4 NP (+1, -1, +r y -r, donde $r = \sqrt{N}$). Con unas funciones

de encaminamiento vertical y horizontal que funcionan dependiendo de que

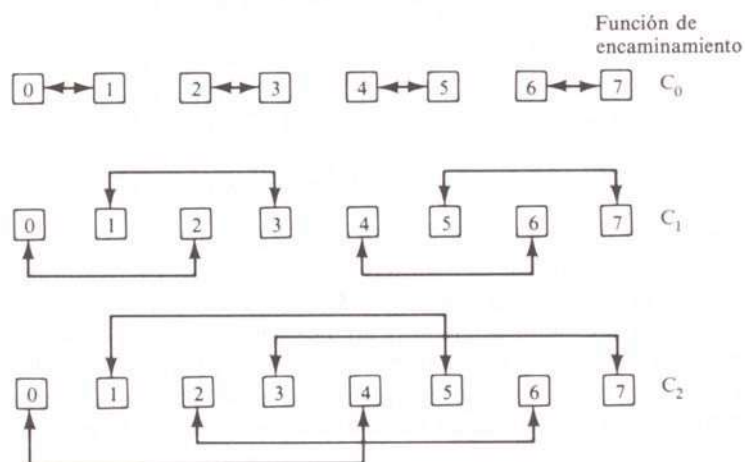
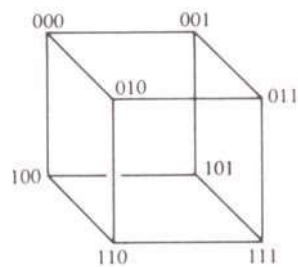
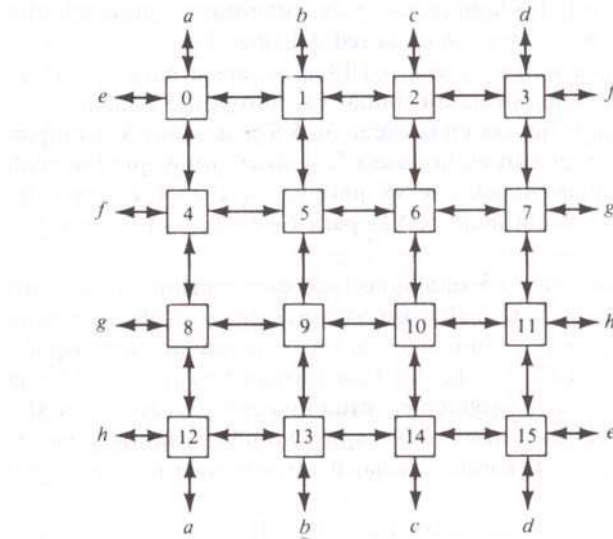
el NP esté activo o no. Topología:



Arquitecturas Paralelas

- Redes de interconexión en cubo:

Se implementa como red recirculante o red multietapa. Dependiendo del número de nodos de procesamiento tendremos un número de bits por nodo y aumentarán las posibilidades de conexión (para un cubo- n , $n = \log_2 N$).



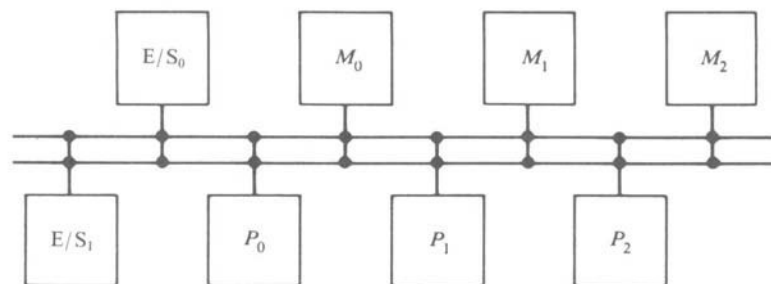
IV.2.- Redes de interconexión MIMD.

Se necesitan dos redes de interconexión, una entre los procesadores y los módulos de memoria y otra entre los procesadores y el subsistema de E/S. La diferencia fundamental respecto a los SIMD es que no existe una unidad de control que gestione todo el proceso. Aquí cada procesador es completo en ese sentido, y esto lleva a que se consideren otras formas físicas disponibles para implementar redes de interconexión, entre ellas están:

- Buses comunes o de tiempo compartido.

Organización de la que hemos comentado algunas características al inicio del tema y que ahora ampliaremos con la consideración multibus y en cómo solucionan este tipo de redes los conflictos de acceso a módulos (algoritmos o mecanismos de arbitraje).

- Multibus: Buses bidireccionales múltiples.



La red de interconexión se convierte en un dispositivo activo que incrementa la complejidad del sistema.

Factores que afectan a las características y rendimiento del bus son: el número de los dispositivos activos conectados al bus; el algoritmo de arbitraje del bus; la centralización o no del control; la longitud de los datos; la sincronización en la transmisión de datos; y la detección de errores.

- Algoritmos de arbitraje: Son relativamente simples, se implementan en hardware y permiten el solapamiento del arbitraje con la transferencia previa. Entre ellos tenemos:

Arquitecturas Paralelas

- Algoritmo de prioridad estática. En caso de peticiones simultáneas el dispositivo con mayor prioridad obtiene el acceso.
 - Algoritmo de intervalo fijo de tiempo. Divide el ancho de banda disponible del bus en unidades de tiempo de longitud fija que se asigna a cada dispositivo de forma rotatoria (TDM).
 - Algoritmos de prioridad dinámica. Intentan equilibrar la carga de los algoritmos anteriores. Dos algoritmos para la permutación dinámica de las prioridades son: LRU (menos recientemente usado), y el RDC (encadenamiento mariposa rotante).
 - Algoritmo FCFS. Primero que llega-primero que se sirve. Se respeta simplemente el orden de recepción de las peticiones. Dificultades de implementación por almacenamiento del orden de llegada de peticiones y los problemas presentados en caso de peticiones simultáneas.
-
- Conmutador de barras cruzadas y memorias multipuerto.

Incrementando el número de buses en tiempo compartido de forma que exista un camino disponible para cada unidad de memoria, tenemos una red de interconexión de barras cruzadas sin bloqueo, con conectividad completa.

Si el control, la conmutación y la lógica de arbitraje de prioridad que están distribuidos por toda la matriz del conmutador de barras cruzadas se distribuyen en los interfaces de los módulos de memoria tendremos una memoria multipuerto. La solución a conflictos en el acceso a los módulos de memoria se realiza asignando prioridades fijas a cada puerto de memoria. La flexibilidad para configurar este tipo de sistemas hace posible la asignación de porciones de memoria privada para ciertos procesadores. Utilizando una topología completamente conectada podemos soportar accesos sin bloqueo a la memoria.

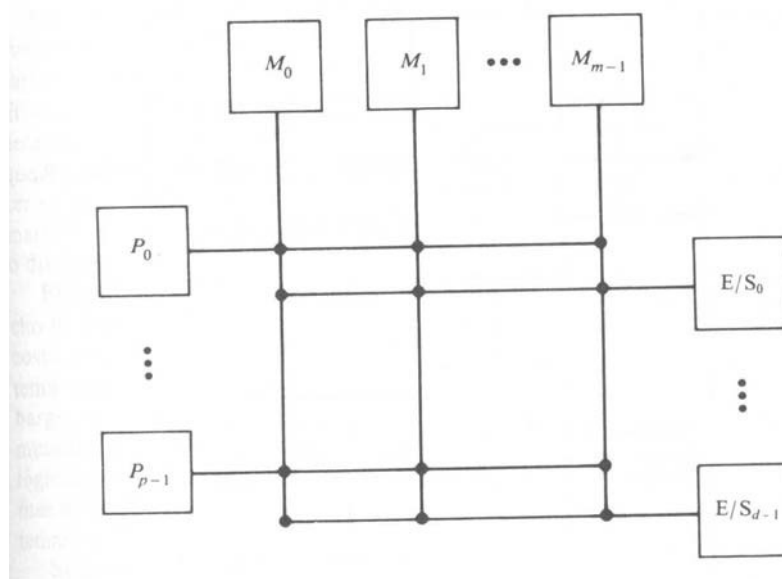
- Barras cruzadas (sin bloqueo, conectividad completa):

Incrementar número de buses en tiempo compartido de forma que exista un camino disponible para cada unidad de memoria.

- Memoria multipuerto:

El control, la conmutación y la lógica de arbitraje de prioridad distribuidos por toda la matriz del conmutador de barras cruzadas.

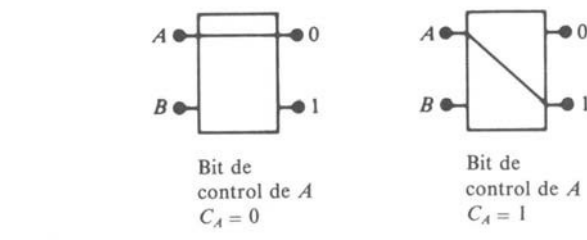
Arquitecturas Paralelas



Solución a conflictos en el acceso a los módulos de memoria: Asignando prioridades fijas a cada puerto de memoria. Flexibilidad de configuración: porciones de memoria privada.

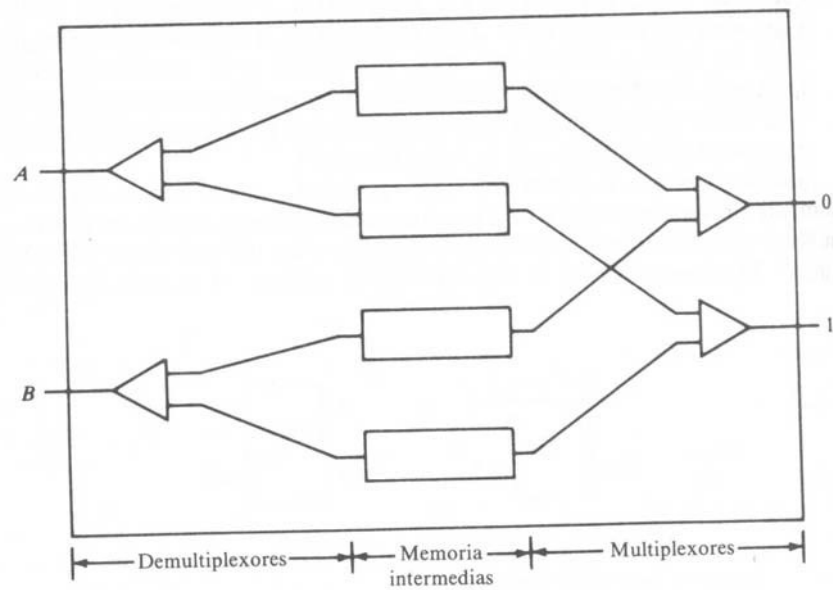
- Redes multietapa para multiprocesadores.

Implementación con cajas de conmutación.

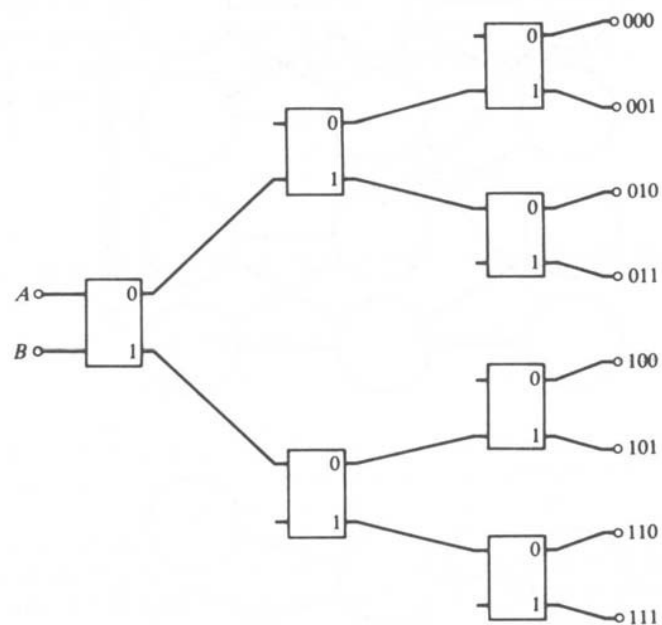


Mejora del rendimiento: Intercalación de memorias intermedias.

Arquitecturas Paralelas



Ejemplos de implementación: red baniano; red delta (baraje); multiplexores y demultiplexores de dispersión; etc.



V.- Sistemas operativos multiprocesador.

GENERALIDADES.

Las funciones de un sistema operativo se resumen:

- Asignación y administración de recursos.
- Protección de datos y tablas.
- Prevención de bloqueos en el sistema.
- Terminación anormal de procesos.

En supercomputadores, para el usuario final un solo sistema operativo controla todos los recursos del sistema. Como en cualquier sistema múltiples tareas o procesos están activos al mismo tiempo y es responsabilidad del sistema operativo planificar su ejecución y la asignación de recursos. En este caso existen además unas **funciones adicionales** que deben soportar para fiabilidad y rendimientos que su costo supone, estas funciones adicionales en sistemas operativos multiprocesadores se resumen:

- Mantener equilibrio de carga de E/S.
- Equilibrio de carga del procesador.
- Reconfiguración del sistema, cuando un procesador falla.

De esta forma los recursos serán utilizados de forma eficiente, de otra forma la inversión en múltiples CPU's se desperdicia.

CLASIFICACIÓN. Tres organizaciones utilizadas:

- Configuración maestro-subordinado:

La rutina ejecutiva se ejecuta siempre en el mismo procesador.

El subordinado necesita servicio, lo pide al supervisor y espera que la ejecución actual en éste sea interrumpida para atender su petición.

S.O. efectivo para aplicaciones especiales con carga de trabajo muy bien definida y en sistemas con subordinados de menores prestaciones.

Arquitecturas Paralelas

Ventajas:

Simplificación de problemas de conflictos y bloqueos.

Necesidades hardware y software simples. Inconvenientes:

Sistema inflexible.

Fallos catastróficos en procesador maestro atendidos por operador.

Tiempo de desocupación de subordinados apreciable si no son adecuadamente atendidos por el maestro.

- Supervisor separado en cada procesador:

Cada procesador tiene una copia del núcleo básico, su propio subsistema de

E/S, archivos, etc; y se sirve sus propias necesidades. Compartición de recursos a un nivel más alto.

Parte del código supervisor duplicado (o reentrante) para permitir copias separadas en cada procesador.

Dificultad de reinicialización de un procesador individual que falle. Intervención manual en reconfiguración de E/S.

- Control supervisor flotante:

El control maestro pasa de un procesador a otro. Varios procesadores pueden ejecutar simultáneamente rutinas de supervisión.

Mejor balanceo de carga en recursos que los anteriores. Resolución de conflictos por prioridades (estáticas o dinámicas).

Código supervisor reentrante (varios procesadores pueden ejecutar la misma rutina de servicio al mismo tiempo).

Exige controlar conflictos de acceso a tablas comunes, para mantener la integridad del sistema.

- EJEMPLO DE MULTIPROCESAMIENTO. IBM S/370.

Múltiples CPU's comparten el almacenamiento principal y se comunican para coordinar actividades.

Copia del S.O. compartida por todas las CPU's.

Cada CPU tiene sus propios canales de E/S, y todas pueden acceder a todos pidiendo que le ejecuten a ella una rutina concreta.

Características claves de este multiprocesamiento:

- Prefijación:

Para compartir áreas críticas de memoria principal.

Reglas de construcción de la dirección absoluta. Objetivos.

- Señalización:

Para comunicación entre procesadores. Instrucción para ello: SIGP

3 operandos: dirección CPU; código de orden (acción solicitada); y estado (dirección registro con resultado instrucción).

- Sincronización:

Para coordinar operaciones conflictivas de CPU.

Instrucciones e interrupciones para ajuste y sincronización de relojes en las CPU's (TOD).

VI.- Programación de supercomputadores.

INTRODUCCIÓN

El desarrollo de programas en supercomputadores se ha diferenciado, históricamente, del desarrollo de programas en computadores de propósito general por la posibilidad de la realización de programación paralela, entendiendo por tal la posibilidad de realizar programas que ejecuten distintas tareas de un mismo programa en distintos procesadores (paralelismo de tareas); o de tratar distintos elementos de la misma estructura de datos en distintos procesadores con el mismo programa (paralelismo de datos). En la actualidad, las arquitecturas paralelas de los computadores de propósito general posibilitan la realización de programas paralelos para aprovechar los recursos hardware disponibles utilizando ambos tipos de paralelismo.

Cada uno de estos paradigmas de programación paralela tiene unas características particulares a la hora de enfocar la paralelización, y cada uno de ellos se adecúa más o menos a la hora de paralelizar un determinado código.

- **PARALELISMO DE TAREAS**

Objetivo: Descomponer el programa a ejecutar en tareas semi-independientes que se asignan ordenadamente a los procesadores.

Tareas: Conjuntos autónomos de sentencias del programa principal. Método habitual en máquinas de memoria compartida.

Sincronización para acceso a datos (evitar colisiones) mediante uso de semáforos.

Inconvenientes:

Dificultad de automatización (cae en desuso).

Número de alternativas de descomposición en tareas.

Complejidad de dependencias de datos y control entre tareas.

Problemas de escalabilidad.

Dificultad de descomposición del problema en gran cantidad de tareas para ejecución simultánea.

Ventaja: Depuración de código “sencilla”

Datos accesibles sin rutinas de comunicación.

- **PARALELISMO DE DATOS**

Objetivo: Descomponer las estructuras de datos del programa y repartir

operaciones sobre ellas en función del procesador que tiene los datos.

Método impuesto como modo más viable de descomposición de problema secuencial en varios paralelos.

Ventajas:

Sencillez de concepto.

Especial interés con grandes volúmenes de datos en aplicaciones para procesamiento paralelo.

Escalabilidad del problema.

Automatización. Modos de programación según quien efectúe la paralelización.

Inconveniente:

Datos vecinos en procesadores distintos.

- **Modos de programación:**

a) Paralelización automática.

El compilador genera de forma automática la versión paralela del código secuencial.

Tradicionalmente ligada a arquitecturas de memoria distribuida. Idea perseguida por investigaciones actuales.

Arquitecturas Paralelas

Resultados poco alentadores. El compilador necesita:

- Saber comportamiento del programa (bucles, saltos, etc.)
- Conocer datos necesarios en la ejecución de iteraciones.
- Dependencias de datos en iteraciones susceptibles de ejecución simultánea.
- En ocasiones cambiar comportamiento del código secuencial: reprogramación de partes de código; modificación de estructuras de datos; cambios en acceso a datos; etc.

Ejemplos: Polaris, Parafrase.

b) Librerías de pase de mensaje.

El programador paraleliza manualmente el código secuencial.

Comunicación entre procesadores mediante el uso de primitivas de paso de mensajes. Un procesador envía datos (send) y otro los recibe (receive).

Filosofía SPMD aplicada a arquitecturas SIMD y MIMD.

Criterios de descomposición determinados por la regla de “que compute el propietario”. Aplicación a bucles -> tamaño iteraciones / N.

Ejecución del programa en etapas:

Computación: datos locales a procesadores. Comunicación: datos no locales.

Uso extendido de este método, caracterizado:

- Control total sobre el hardware.
- Buena eficiencia.
- Elevado coste (desarrollo y depuración).

Dificultades de programación: imposibilidad de control mental de ejecución simultánea y complejidad de sincronización y formato de las comunicaciones.

Arquitecturas Paralelas

c) Lenguaje de paralelismo de datos.

Se impone como estándar en programación paralela.

Objetivo: Producir código paralelo eficiente en poco tiempo.

Explotar ventajas de los anteriores.

Uso de espacio de direcciones global.

Preservar flujo secuencial y dar al compilador el trabajo de más bajo nivel.

Insertar rutinas de comunicación.

Método: Incorporar al código secuencial directivas que informan al compilador sobre los datos y computaciones distribuidos.

- Directivas del lenguaje de paralelismo de datos:

1.- Nuevas directivas: Informan al compilador de propiedades del código o sugerencias de descomposición del código secuencial. Ej.: DISTRIBUTE, ALIGN, INDEPENDENT

2.- Extensiones del lenguaje: Elementos nuevos que lo amplían y modifican la interpretación del mismo. Ej.: FORALL

3.- Librerías intrínsecas: Conjuntos de rutinas muy valiosas en computación paralela. Ej.: Operaciones de reducción, ordenación, reparto y recogida de datos, etc.

Alternativa muy atractiva.

EFICIENCIA	PROGRAMACIÓN	FACILIDAD DE USO
- +		- +
1 2 3 4 5 6 7 8 9 10		1 2 3 4 5 6 7 8 9 10
X X	MANUAL	X X X
X X	PARAL. DE DATOS	X X
X X	AUTOMÁTICA	X

- EJEMPLO DE PROGRAMACIÓN

Basado en paralelismo de datos.

Transformaciones en programa secuencial, para obtener código SPMD.

Compilación individual en cada nodo, carga y ejecución simultánea sobre distintos datos.

Caso de estudio: Multiplicación de dos matrices cuadradas.

Pasos del proceso de paralelización:

1.- Seleccionar la distribución de datos a utilizar.

Paso con más impacto en la eficiencia del código objeto. Determina la localidad de referencia en el acceso a datos.

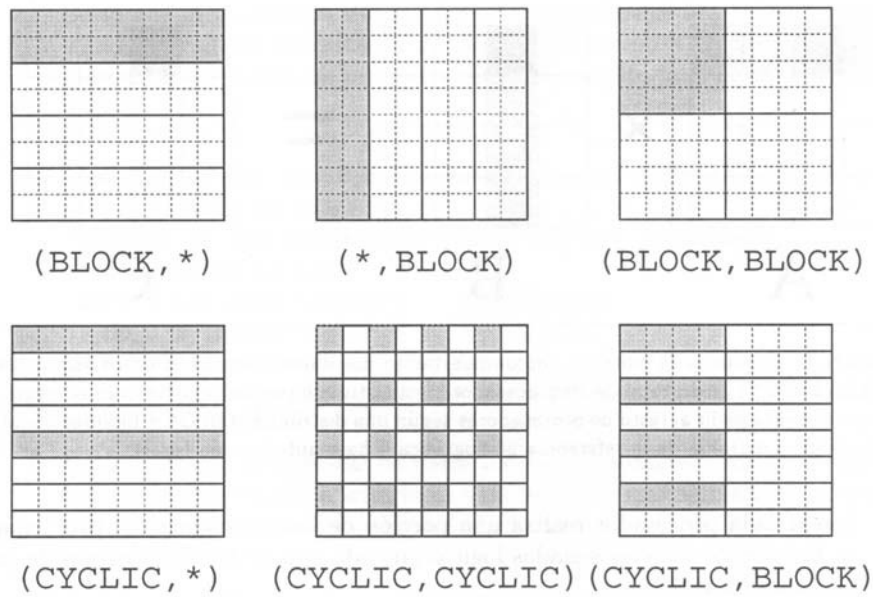
Las más usadas:

BLOCK. Descompone el vector en subvectores de datos consecutivos. CICLYC. Reparte los datos entre los procesadores individualmente de forma

repetitiva.

Distribuciones replicadas. Parcial (un dato en varios procesadores); Total (un dato a todos los procesadores).

Ej.: Matriz bidimensional en 4 procesadores

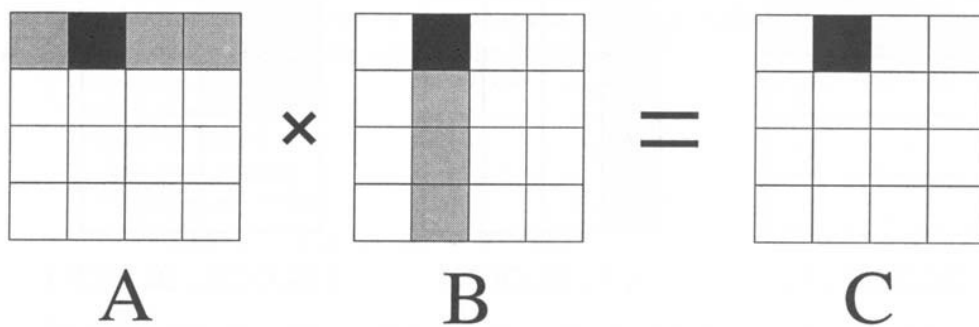


Diferencias comunicaciones BLOCK y replicación parcial de filas y columnas en problema de multiplicación.

Nuestro ejemplo:

16 nodos procesadores, en matriz de 4x4. Distribución (BLOCK,BLOCK).

Caso procesador (1,2)



2.- Descomposición de los bucles del algoritmo.

Reparto de iteraciones entre procesadores. Según distribución de datos

Según regla de “computa el propietario”. En nuestro ejemplo: Código original:

```
Do i=1,1000
```

```
Do j=1,1000
```

```
Do k=1,1000
```

```
C[i][j] =+ A[i][k]*B[k][j]
```

Código modificado:

```
Do i = (mi_fila - 1)*250 + 1, mi_fila*250
```

```
Do j = (mi_col - 1)*250 + 1, mi_col*250
```

```
Do k=1,N
```

```
C[i][j] =+ A[i][k]*B[k][j]
```

3.- Inserción de comunicaciones interprocesador.

Analizar datos no locales que cada computación necesita. Nuestro ejemplo quedaría:

Recibir de los proc's de mi misma fila su matriz local de A y enviarle la mía.

Recibir de los proc's de mi misma columna sus datos de B y enviarle la mía.

```
Do i=(mi_fila - 1)*250 + 1, mi_fila*250
```

```
Do j=(mi_col - 1)*250 + 1, mi_col*250
```

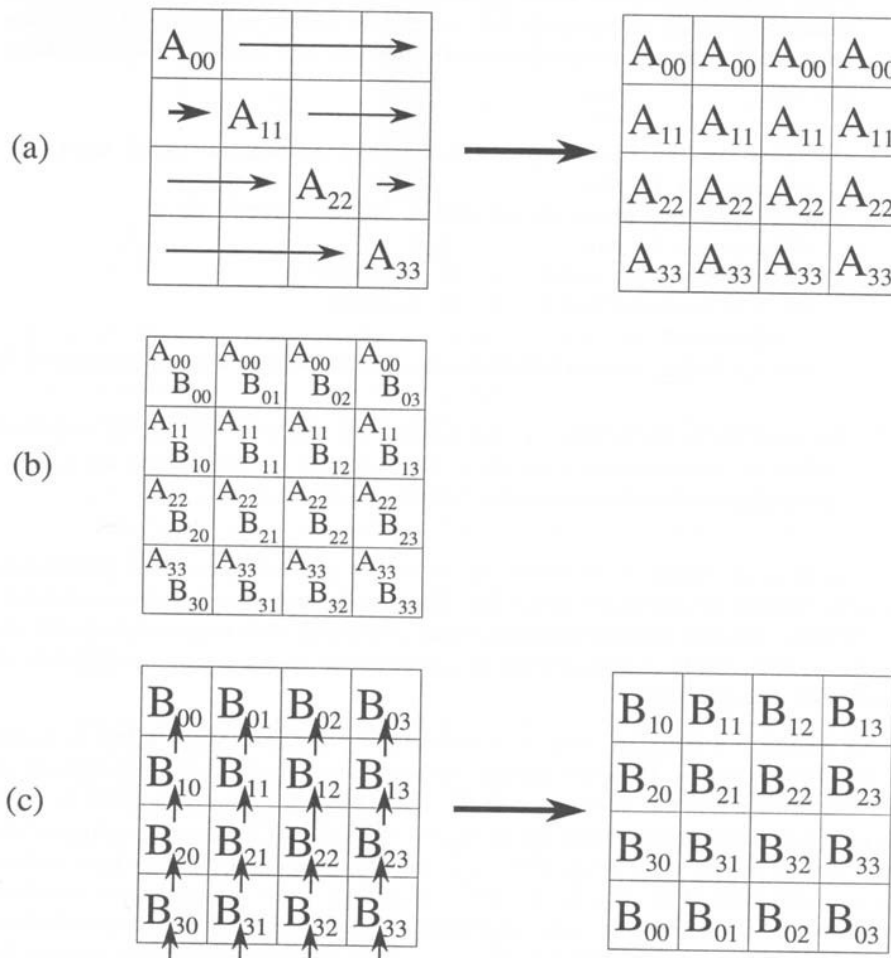
```
Do k=1,N
```

```
C[i][j] =+ A[i][k]*B[k][j]
```

4. Declaración de estructuras de datos locales y variables auxiliares.

Esquema de ejecución del algoritmo sobre la malla de 4x4:

Arquitecturas Paralelas



Para el caso de "Lenguaje de paralelismo de datos":

```
PROGRAM PROCESSORS(4,4) PARAMETER (N=1000)
```

```
REAL A(N,N), B(N,N), C(N,N) DISTRIBUTE A, B, C: (BLOCK,BLOCK)
```

```
DO I=1,N INDEPENDENT
```

```
DO J=1,N INDEPENDENT
```

```
DO K=1,N INDEPENDENT
```

```
  C[I][J]=A[I][K]*B[K][J]
```

```
END
```